

**UNIVERSIDADE FEDERAL DE SERGIPE**

Disciplina: Sistemas Operacionais — 2026.2

## **Atividade 1 (AV1)**

*Processos, Threads, Escalonamento e Inferência Local com Ollama*

[https://youtu.be/JfuxJnVD\\_vU](https://youtu.be/JfuxJnVD_vU)

### **Integrantes:**

Bruno Saint Clair Silva Oliveira

Gabriel Bastos Pimentel

João Victor Oliveira Moura

José Cerqueira da Silva Neto

Leonardo de Souza Aragão

Marcus Antônio Martins Oliveira Júnior

Yan Victor Araujo do Nascimento

# 1. Tabela indicativa das contribuições da equipe

| Integrante                             | Papel                                                                                       | Principais atividades realizadas                                                                                                                                                                    |
|----------------------------------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bruno Saint Clair Silva Oliveira       | Modelo, Experimentos, Processos/Threads e Relatório                                         | Participei da escolha do modelo e da camada de aplicação, realizei todos os testes/experimentos documentando os resultados e preenchi as seções 8, 11 e 14 do relatório.                            |
| Gabriel Bastos Pimentel                | Experimentos e Relatório                                                                    | Participei do monitoramento e registro dos testes realizados na aplicação RAG, também colaborei na revisão, organização do relatório                                                                |
| João Victor Oliveira Moura             | Modelo e Relatório                                                                          | Ajudei na escolha do modelo que usamos para realizar os testes. Ajustei algumas coisas do relatório.                                                                                                |
| José Cerqueira da Silva Neto           | Criação e modelagem da estrutura do relatório. Redação da introdução e motivações iniciais. | Construção estrutural do relatório e dos tópicos de introdução. Descrição da camada de aplicação.                                                                                                   |
| Leonardo de Souza Aragão               | Modelo e Apresentação                                                                       | Realizei testes no modelo, estruturação da apresentação e material de apoio.                                                                                                                        |
| Marcus Antônio Martins Oliveira Júnior | Relatório                                                                                   | Auxiliei no acompanhamento e documentação dos testes de execução da aplicação RAG e coleta de métricas. Além de colaborar na revisão, estruturação e redação final das seções do relatório técnico. |
| Yan Victor Araujo do Nascimento        | Experimentos e Relatórios                                                                   | Realizei os experimentos e a coleta das métricas de desempenho da aplicação, além de colaborar na análise e organização dos resultados apresentados no relatório.                                   |

## 2. Introdução e problema

A execução local de sistemas de Inteligência Artificial generativa permite analisar, de forma prática, a relação entre aplicações de IA e os recursos disponibilizados pelo Sistema Operacional (SO). Diferentemente de serviços de IA executados em servidores remotos, uma aplicação local precisa utilizar diretamente os recursos de hardware da máquina, como processador, memória RAM, armazenamento e, quando disponível, GPU.

Neste trabalho é analisada uma aplicação de RAG (Retrieval-Augmented Generation) para documentos PDF, desenvolvida a partir do projeto disponibilizado no repositório da equipe. A aplicação integra Ollama, LangChain, FastAPI e Next.js, permitindo carregar documentos PDF e realizar perguntas sobre seu conteúdo. O processamento é realizado localmente, utilizando o modelo LLM LiquidAI LFM2.5-2.6B na quantização Q4\_0 para geração das respostas e o modelo nomic-embed-text para geração dos embeddings. Essas escolhas estão definidas no próprio repositório do projeto.

O funcionamento da aplicação envolve diferentes etapas computacionais. Inicialmente, o documento PDF é carregado e seu conteúdo é processado. Em seguida, o texto é dividido e transformado em representações vetoriais por meio do modelo de embeddings. Essas representações são armazenadas em uma base vetorial, permitindo que, quando o usuário realize uma pergunta, sejam recuperados os trechos mais relevantes do documento. O contexto recuperado é então enviado ao modelo de linguagem, que produz a resposta. Esse fluxo permite observar diferentes comportamentos do Sistema Operacional. Durante o processamento dos documentos, há operações de leitura e escrita no armazenamento, utilização de memória e processamento para extração, divisão e indexação do conteúdo. Durante uma consulta, ocorre uma combinação entre recuperação de informações, geração de embeddings e inferência do modelo de linguagem. No ambiente analisado, esses modelos são executados localmente pelo Ollama, permitindo observar diretamente o consumo de recursos da máquina.

O problema investigado consiste em analisar o comportamento dos recursos do Sistema Operacional durante a execução de uma aplicação RAG local, identificando como as diferentes etapas do processamento afetam CPU, memória, armazenamento, processos e threads. Busca-se também diferenciar o comportamento durante a indexação dos documentos e durante a consulta e geração das respostas, verificando quais componentes são responsáveis pelo consumo de recursos em cada etapa. Além disso, pretende-se analisar

os efeitos do tamanho e da quantidade de documentos processados. Dessa forma, a aplicação RAG funciona como um estudo de caso para relacionar conceitos de Sistemas Operacionais — como processos, threads, gerenciamento de memória, entrada e saída e chamadas de sistema — com uma aplicação moderna de Inteligência Artificial executada localmente.

### 3. Trilha e camada de aplicação selecionadas

A equipe selecionou a Trilha C – RAG sobre PDF: Ollama + ollama\_pdf\_rag, cuja camada de aplicação é implementada com Ollama, LangChain, FastAPI e Next.js, com uma interface adicional em Streamlit disponível como opção de visualização. O backend em FastAPI expõe a lógica de indexação e consulta dos documentos, enquanto o frontend em Next.js fornece a interface web para o usuário enviar PDFs e realizar perguntas sobre seu conteúdo.

A escolha dessa trilha se justifica por permitir observar, em um único sistema, praticamente todas as camadas de execução previstas na atividade: um processo de backend (FastAPI/Python), um processo de frontend (Next.js/Node.js), um serviço de inferência local (Ollama) e, opcionalmente, uma interface adicional (Streamlit). Essa composição de múltiplos processos e linguagens de execução torna a trilha interessante para a disciplina de Sistemas Operacionais, pois viabiliza a análise de conceitos como múltiplos processos independentes e seu consumo de recursos, a interação entre indexação e inferência que produzem padrões distintos de uso de CPU e memória, assim como concorrência e escalonamento de processos que exigem do sistema operacional gerenciamento de filas e alocação de CPU.

Dessa forma, a Trilha C conecta diretamente os conceitos centrais da disciplina a um cenário realista de uma aplicação de IA generativa local com arquitetura de múltiplos serviços, reforçando o objetivo da atividade de relacionar a teoria abordada em sala de aula com a prática de uma aplicação funcional.

#### 3.1 Ficha da camada de aplicação

- **Nome e URL do repositório GitHub:** Chat with PDF locally using Ollama + LangChain; [https://github.com/tonykipkemboi/ollama\\_pdf\\_rag](https://github.com/tonykipkemboi/ollama_pdf_rag).
- **Autor / organização:** Tony Kipkemboi.
- **Licença:** MIT License.
- **Commit, tag ou release utilizado:** Commit: 206749ed103e6fa4683a3a89b1abd4a3034febb2; Branch: Main.
- **Data de acesso:** 13/09/2026.
- **Finalidade da aplicação:** Aplicação de RAG (Retrieval-Augmented Generation) para realizar consultas em documentos PDF. A aplicação extrai e indexa o conteúdo dos PDFs, recupera trechos relevantes para uma pergunta e utiliza uma LLM local executada pelo Ollama para gerar respostas baseadas no conteúdo dos documentos.
- **Dependências principais:**

```
# Backend Python
ollama>=0.6.0
streamlit==1.40.0
pdfplumber>=0.11.8
pdfminer.six==20251107
langchain==1.0.0
langchain-core>=1.0.0
langchain-ollama==1.0.1
langchain_community>=0.4.1
langchain_text_splitters==1.0.0
langchain-classic==1.0.0
langchain-chroma>=0.2.0
unstructured>=0.16.12
unstructured[all-docs]>=0.16.12
onnx>=1.17.0
protobuf>=5.29.5
```

chromadb>=0.4.22

Pillow==10.4.0

numpy==1.26.4

pytest>=8.2,<9

pytest-cov==4.1.0

coverage==7.4.0

pydantic==2.10.4

#### # Dependências FastAPI

fastapi>=0.104.0

uvicorn[standard]>=0.24.0

python-multipart>=0.0.6

sqlalchemy>=2.0.0

aiosqlite>=0.19.0

pydantic-settings>=2.0.0

#### # Dependências Front-End

ai-sdk/gateway: 2.0.18,

ai-sdk/provider: 2.0.0,

ai-sdk/react: 2.0.109,

codemirror/lang-javascript: 6.2.3,

codemirror/lang-python: 6.1.7,

codemirror/state: 6.5.2,

codemirror/theme-one-dark: 6.1.2,

codemirror/view: 6.36.4,

icons-pack/react-simple-icons: 13.8.0,

opentelemetry/api: 1.9.0,

opentelemetry/api-logs: 0.200.0,

radix-ui/react-collapsible: 1.1.12,

radix-ui/react-dialog: 1.1.15,

radix-ui/react-dropdown-menu: 2.1.16,

radix-ui/react-hover-card: 1.1.15,

radix-ui/react-icons: 1.3.2,

radix-ui/react-progress: 1.1.8,

radix-ui/react-scroll-area: 1.2.10,

radix-ui/react-select: 2.2.6,

radix-ui/react-separator: 1.1.8,

radix-ui/react-slot: 1.2.4,

radix-ui/react-tooltip: 1.2.8,

radix-ui/react-use-controllable-state: 1.2.2,

radix-ui/react-visually-hidden: 1.1.2,

vercel/analytics: 1.5.0,

vercel/blob: 0.24.1,

vercel/functions: 2.2.13,

vercel/otel: 1.14.0,

xyflow/react: 12.10.0,

ai: 5.0.108,  
bcrypt-ts: 5.0.3,  
class-variance-authority: 0.7.1,  
classnames: 2.5.1,  
clsx: 2.1.1,  
cmdk: 1.1.1,  
codemirror: 6.0.1,  
date-fns: 4.1.0,  
diff-match-patch: 1.0.5,  
dotenv: 16.4.7,  
drizzle-orm: 0.34.1,  
embla-carousel-react: 8.6.0,  
fast-deep-equal: 3.1.3,  
framer-motion: 11.18.2,  
geist: 1.3.1,  
katex: 0.16.25,  
lucide-react: 0.446.0,  
motion: 12.23.26,  
nanoid: 5.1.3,  
next: 16.0.10,  
next-auth: 5.0.0-beta.30,  
next-themes: ^0.3.0,  
orderedmap: ^2.1.1,  
papaparse: 5.5.2,  
postgres: 3.4.5,  
prosemirror-example-setup: 1.2.3,  
prosemirror-inputrules: 1.4.0,  
prosemirror-markdown: 1.13.1,  
prosemirror-model: 1.24.1,  
prosemirror-schema-basic: 1.2.3,  
prosemirror-schema-list: 1.5.1,  
prosemirror-state: 1.4.3,  
prosemirror-view: 1.38.1,  
radix-ui: 1.4.3,  
react: 19.0.1,  
react-data-grid: 7.0.0-beta.47,  
react-dom: 19.0.1,  
react-resizable-panels: 2.1.7,  
react-syntax-highlighter: 15.6.6,  
redis: 5.9.0,  
resumable-stream: 2.2.8,  
server-only: 0.0.1,  
shiki: 3.14.0,  
sonner: 1.7.4,  
streamdown: 1.4.0,  
swr: 2.3.3,

tailwind-merge: 2.6.0,  
tailwindcss-animate: 1.0.7,  
use-stick-to-bottom: 1.1.1,  
usehooks-ts: 3.1.1,  
zod: 3.25.76

- **Arquitetura Simplificada:** A arquitetura da aplicação é organizada em camadas. A interface, desenvolvida com Next.js ou Streamlit, permite ao usuário enviar PDFs e realizar consultas. O backend em FastAPI recebe essas requisições e coordena o processamento dos documentos e das consultas. No processo de indexação, o conteúdo dos PDFs é extraído, dividido em trechos e transformado em embeddings pelo modelo nomic-embed-text, sendo armazenado no ChromaDB. Durante uma consulta, o sistema recupera os trechos mais relevantes e os envia, juntamente com a pergunta do usuário, ao Ollama, que executa o modelo LFM2.5-2.6B Q4\_0 para gerar a resposta. O SQLite também é utilizado para armazenar dados da aplicação e do histórico de conversas.

- **Comandos de instalação:**

```
# git clone https://github.com/tonykipkemboi/ollama_pdf_rag.git
# cd ollama_pdf_rag
# python3.11 -m venv venv
# source venv/bin/activate
# pip install -r requirements.txt
# cd web-ui
# pnpm install
# cd ..
# ollama pull hf.co/LiquidAI/LFM2.5-2.6B-GGUF:Q4_0
# ollama pull nomic-embed-text
```

- **Comandos de execução:**

```
# Terminal 1
source venv/bin/activate
python run_api.py
```

```
# Terminal 2
cd web-ui
pnpm dev
```

- **Erros encontrados:**

```
#MissingSecret no Auth.js: a interface Web não possuía a variável AUTH_SECRET configurada no arquivo .env;
#Erro de migração do banco SQLite/Drizzle: a migração tentou utilizar gen_random_uuid, função incompatível com a
configuração SQLite utilizada.
#Erro do numba/coverage: durante o upload e processamento de PDF ocorreu incompatibilidade envolvendo
coverage.types/Tracer, impedindo o processamento do documento.
#UND_ERR_HEADERS_TIMEOUT no frontend: a interface Web atingia timeout ao aguardar a resposta do backend.
# Erro 404 em /favicon.ico: o backend não possuía um favicon configurado. Esse erro não afetou o funcionamento da
aplicação.
```

## 4. Modelo Hugging Face selecionado

- **Nome completo do modelo:** LiquidAI/LFM2.5-2.6B.
- **URL do model card (Hugging Face):** <https://huggingface.co/LiquidAI/LFM2.5-2.6B>; URL da quantização: <https://huggingface.co/LiquidAI/LFM2.5-2.6B-GGUF>.
- **Família / arquitetura:** LFM2.5 / LFM2 (arquitetura híbrida, combinando blocos de convolução curta com GQA — Grouped Query Attention). O modelo possui 30 camadas: 22 blocos de convolução e 8 blocos GQA.
- **Checkpoint / variante:** LFM2.5-2.6B, modelo pós-treinado (instruction-tuned).
- **Número de parâmetros:** 2,69 bilhões de parâmetros (2.69B).
- **Formato:** GGUF;
- **Quantização:** Q4;
- **Licença:** LFM Open License v1.0 (lflm1.0).
- **Janela de contexto máxima:** 131.072 tokens (128K).
- **Tamanho dos arquivos baixados:** 1,6 GB.

- **Justificativa da escolha:** O LFM2.5-2.6B foi escolhido por ser um modelo adequado para execução local. Possui 2,6 bilhões de parâmetros, atendendo ao limite proposto na atividade e a versão GGUF com quantização Q4\_0 reduz o consumo de memória e armazenamento. Sua utilização com Ollama e RAG de documentos PDF permitirá analisar métricas como uso de CPU, memória, processos e threads durante as consultas.

## 5. URL do projeto público no GitHub

[https://github.com/BrunoSaintClair/SO\\_UFS\\_2026\\_2\\_Bruno\\_SaintClair\\_Jose\\_Cerqueira](https://github.com/BrunoSaintClair/SO_UFS_2026_2_Bruno_SaintClair_Jose_Cerqueira)

## 6. Ambiente experimental

Inventário do ambiente onde os experimentos foram executados.

**Ambiente de Execução:** Nativo (Bare-metal) — Laptop Acer Nitro ANV15-51.

**Distribuição e versão do SO:** Ubuntu 24.04.3 LTS (Noble Numbat).

**Versão do kernel:** Linux 7.0.0-31-generic (x86\_64).

**Processador:** 13th Gen Intel(R) Core(TM) i5-13420H.

**Núcleos/threads:** 8 núcleos físicos (4 P-Cores + 4 E-Cores) / 12 threads.

**Memória RAM:** 23 GiB Total (~12 GiB disponíveis, 4.0 GiB Swap).

**GPU Dedicada:** NVIDIA GeForce RTX 3050 6GB Laptop GPU.

**VRAM:** 6144 MiB (6 GiB GDDR6) | Driver 595.84 | CUDA 13.2.

**Armazenamento:** SSD NVMe 512 GB (5.3GB livres)

**Versão do Ollama:** v0.32.0

**Modelos de IA:** [hf.co/LiquidAI/LFM2.5-2.6B-GGUF-Q4\\_0](https://huggingface.co/LiquidAI/LFM2.5-2.6B-GGUF-Q4_0) (1.6 GB); nomic-embed-text:latest (274 MB)

## 7. Arquitetura da aplicação

A arquitetura da aplicação pode ser descrita como um sistema dividido em camada de interface, backend/API, processamento RAG, armazenamento e serviço de inferência, tendo o usuário como ponto de entrada.

O usuário interage com a aplicação principalmente de duas formas: enviando documentos PDF para serem processados ou fazendo perguntas sobre os documentos previamente inseridos. A aplicação disponibiliza duas possibilidades de interface: uma interface web desenvolvida com Next.js/React, executada na porta 3000, e uma interface alternativa desenvolvida com Streamlit, normalmente executada na porta 8501. Essas interfaces permitem funcionalidades como conversação com o sistema, envio de PDFs, seleção de documentos e visualização do histórico. Quando o usuário realiza uma operação, a interface se comunica com o backend da aplicação, implementado utilizando FastAPI e Uvicorn, normalmente disponível na porta 8001. O backend funciona como intermediário entre a interface, o processamento dos documentos, o banco de dados, o armazenamento vetorial e o serviço de inteligência artificial.

O backend disponibiliza diferentes rotas da API para realizar essas operações. Entre elas estão as rotas responsáveis pelo upload e gerenciamento dos PDFs, pela realização das consultas RAG, pela obtenção dos modelos disponíveis no Ollama e pela verificação da saúde dos serviços. Uma consulta ao endpoint /query, por exemplo, recebe a pergunta do usuário e inicia o processo de recuperação das informações relevantes antes de solicitar a geração da resposta.

No backend encontra-se a camada de processamento RAG (Retrieval-Augmented Generation). Quando um PDF é enviado, seu conteúdo é extraído e dividido em partes menores. Esses trechos são transformados em representações numéricas chamadas embeddings. Os embeddings permitem representar semanticamente o conteúdo dos documentos e posteriormente realizar buscas por similaridades, sendo gerados utilizando o modelo nomic-embed-text, executado através do Ollama. Os vetores resultantes são armazenados no ChromaDB, que funciona como o banco de dados vetorial da aplicação. Esse armazenamento permite que, quando o usuário fizer uma pergunta, o sistema encontre os trechos dos documentos que possuem maior relação semântica com a pergunta.

Durante uma consulta, o sistema recebe a pergunta do usuário e realiza uma busca no ChromaDB. Os chunks considerados mais relevantes são recuperados e utilizados para construir o contexto da consulta. Esse contexto é combinado com a pergunta original e enviado ao modelo de linguagem responsável pela geração da resposta. A geração da resposta é realizada pelo Ollama, que disponibiliza uma API local, normalmente na porta 11434. No ambiente utilizado pelo projeto, o modelo de linguagem escolhido é o

LFM2.5-2.6B em quantização Q4\_0. O Ollama recebe a pergunta juntamente com o contexto recuperado pelo RAG e executa a inferência para produzir a resposta.

A aplicação também possui uma camada de armazenamento local. Os documentos PDF ficam armazenados no sistema de arquivos, enquanto os dados vetoriais utilizados pelo ChromaDB ficam associados ao armazenamento de vetores da aplicação. Além disso, o projeto utiliza SQLite para armazenar informações como histórico e metadados relacionados à aplicação.

## 8. Processos, threads e chamadas de sistema

### 8.1 Processos e threads observados

A aplicação RAG executa múltiplos processos independentes que cooperam para realizar a indexação dos documentos e a geração das respostas. Os principais componentes observados foram o backend em FastAPI, o frontend em Next.js e o serviço Ollama, responsável pela execução dos modelos de linguagem e de embeddings. A comunicação entre esses componentes ocorre por meio de sockets TCP locais.

Durante os experimentos, foram identificados processos dedicados à inferência do modelo LFM2.5-2.6B, à geração de embeddings com o modelo nomic-embed-text e aos serviços de interface e API. O sistema também utilizou diversas threads para atividades como processamento assíncrono, acesso ao banco vetorial ChromaDB, gerenciamento de banco SQLite e comunicação com a GPU.

No pico de utilização, a aplicação executou mais de 170 threads simultâneas distribuídas entre os diferentes componentes. O escalonador do Linux distribuiu dinamicamente essas threads pelos núcleos disponíveis, sem necessidade de configuração manual de afinidade. Os maiores consumos de CPU e memória foram observados nos processos de inferência executados pelo Ollama, enquanto o frontend apresentou maior utilização de memória RAM devido ao ambiente de desenvolvimento do Next.js.

### 8.2 Chamadas de sistema analisadas

A análise com o strace registrou aproximadamente 16 mil chamadas de sistema durante uma sessão completa da aplicação, incluindo processamento de PDF e consultas ao modelo de IA.

As chamadas relacionadas à criação e sincronização de threads (clone3, futex e sched\_yield) foram predominantes, refletindo o caráter concorrente da aplicação. Também foram observadas chamadas de acesso a arquivos (openat, read, lseek e mmap), utilizadas na leitura dos documentos PDF, carregamento de bibliotecas e gerenciamento de memória.

Por fim, chamadas de rede como socket, connect, sendto e recvfrom demonstraram a comunicação entre o backend FastAPI e o serviço Ollama por meio da interface de loopback local. Esses resultados evidenciam a utilização conjunta de mecanismos de concorrência, gerenciamento de memória, acesso a arquivos e comunicação entre processos para sustentar o funcionamento do pipeline RAG.

## 9. Configurações e metodologia

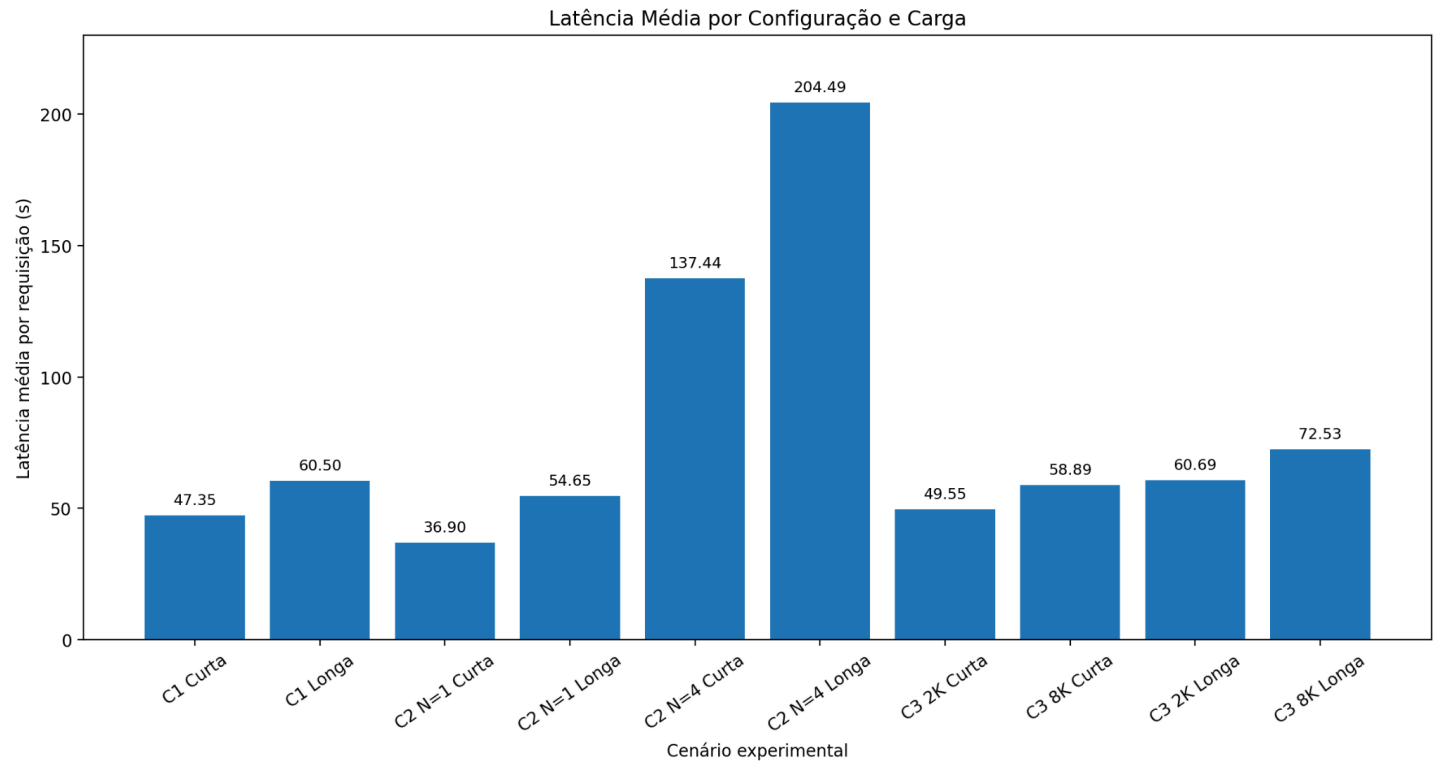
Foram avaliadas três configurações experimentais. A primeira corresponde à execução padrão da aplicação utilizando os parâmetros originais do projeto, servindo como linha de base para comparação. A segunda investigou o impacto da concorrência, comparando execuções com uma única requisição (N=1) e quatro requisições simultâneas (N=4). A terceira avaliou o efeito do tamanho da janela de contexto, comparando execuções com 2.048 e 8.192 tokens.

Os experimentos utilizaram o documento *CaféComCase\_LAWD.pdf* e dois tipos de consulta: uma pergunta curta e uma pergunta longa. Cada cenário foi repetido três vezes, permitindo comparar latência, consumo de recursos e estabilidade da aplicação em diferentes condições de execução.

# 10. Resultados

Preencher a tabela de resultados abaixo e insira, no mínimo, um gráfico ou visualização equivalente.

| Configuração           | Cenário / Entrada     | Repetição | Tempo inicialização | TTFT      | Latência total  | CPU média (%) | RAM média após |
|------------------------|-----------------------|-----------|---------------------|-----------|-----------------|---------------|----------------|
| 1 - Padrão             | Pergunta curta        | 3         | 3,9461 s            | 47,3449 s | 47,3450 s       | 7,86%         | 11.928 MiB     |
| 1 - Padrão             | Pergunta longa        | 3         | 3,9461 s            | 60,4983 s | 60,4984 s       | 4,38%         | 12.120 MiB     |
| 2 - Concorrência/Carga | N=1 – curta           | 3         | Modelo já carregado | 36,8973 s | 36,8975 s       | 6,73%         | 12.326 MiB     |
| 2 - Concorrência/Carga | N=1 – longa           | 3         | Modelo já carregado | 54,6473 s | 54,6475 s       | 8,17%         | 12.457 MiB     |
| 2 - Concorrência/Carga | N=4 – curta           | 3         | Modelo já carregado | 137,44 s  | 137,4401 s/req. | 7,47%         | 12.919 MiB     |
| 2 - Concorrência/Carga | N=4 – longa           | 3         | Modelo já carregado | 204,49 s  | 204,4900 s/req. | 7,72%         | 13.906 MiB     |
| 3 – Ajuste técnico     | Contexto 2048 – curta | 3         | 7,1519 s            | 49,5546 s | 49,5547 s       | 3,78%         | 12.574 MiB     |
| 3 – Ajuste técnico     | Contexto 2048 – longa | 3         | 7,1519 s            | 60,6856 s | 60,6857 s       | 2,99%         | 12.489 MiB     |
| 3 – Ajuste técnico     | Contexto 8192 – curta | 3         | 7,6281 s            | 58,8899 s | 58,8900 s       | 2,99%         | 12.600 MiB     |
| 3 – Ajuste técnico     | Contexto 8192 – longa | 3         | 7,6281 s            | 72,5257 s | 72,5259 s       | 3,29%         | 13.075 MiB     |



## 11. Discussão

Os experimentos demonstraram que houve competição significativa por recursos sob carga concorrente. Na Configuração 2, ao escalar Os resultados indicam que o principal fator limitante da aplicação é o processo de inferência executado na GPU. Embora múltiplas requisições possam ser recebidas simultaneamente, elas tendem a ser atendidas em fila, aumentando o tempo de espera sob carga concorrente.

A utilização de janelas de contexto maiores também impactou diretamente o consumo de memória e a latência, evidenciando o custo computacional associado ao armazenamento de informações adicionais para o modelo. Em contrapartida, a estabilidade da taxa de geração de tokens sugere que o aumento de custo está concentrado principalmente na preparação do contexto e não na etapa de geração das respostas.

De forma geral, os resultados confirmam que aplicações RAG locais apresentam forte dependência dos recursos de memória e processamento disponíveis, especialmente durante a etapa de inferência.

## 12. Relação com Sistemas Operacionais e IA generativa

A execução da aplicação permitiu observar diversos conceitos estudados em Sistemas Operacionais. O escalonador do Linux distribuiu centenas de threads entre os núcleos disponíveis, mantendo a aplicação responsiva mesmo durante períodos de maior carga.

A análise das chamadas de sistema evidenciou o uso de mecanismos de sincronização, gerenciamento de memória, acesso a arquivos e comunicação entre processos. Além disso, a interação entre RAM, VRAM e armazenamento demonstrou a importância do gerenciamento de memória em aplicações modernas de IA.

Dessa forma, a aplicação RAG serviu como um estudo de caso prático para relacionar conceitos de processos, threads, escalonamento, memória virtual, entrada e saída e comunicação entre processos com a execução local de modelos de linguagem generativa.

## 13. Limitações e ameaças à validade

Os resultados obtidos estão condicionados ao ambiente utilizado nos experimentos, composto por um notebook com processador Intel i5, GPU RTX 3050 e Ubuntu Linux. Alterações no hardware, sistema operacional ou modelos de IA podem produzir comportamentos diferentes.

Além disso, os testes foram realizados utilizando apenas um documento PDF e uma carga máxima de quatro requisições simultâneas. Cenários com bases documentais maiores, modelos mais robustos ou níveis superiores de concorrência podem apresentar padrões distintos de consumo de recursos e desempenho.

Também deve ser considerado que ferramentas de monitoramento, como o strace, introduzem pequena sobrecarga durante a coleta de dados, podendo influenciar levemente os tempos observados. Por fim, a natureza probabilística dos modelos de linguagem gera variações naturais entre execuções, mesmo quando as condições experimentais permanecem constantes.

Essa versão mantém o caráter técnico e acadêmico do relatório, mas reduz em torno de **40% a 50% do texto original**, que costuma ser um ponto ideal para trabalhos de graduação.

## 14. Declaração de uso de IA generativa

**Ferramenta e modelo de IA utilizados:** Antigravity CLI com os modelos Claude Opus 4.6 e Gemini 3.8 Flash.

**Finalidade de uso 1:** Ajudar a rodar a aplicação e resolver erros na subida do backend (FastAPI), frontend (Next.js) e banco SQLite.

**Finalidade de uso 2:** Ajudar a montar os scripts de teste e os comandos de monitoramento no Linux para o relatório.

**Prompts relevantes utilizados (até 5):**

1. "Como resolver erro no Next.js dizendo que falta a variável AUTH\_SECRET e erro de migração no SQLite?"
2. "Faz um script bash para rodar o backend e o frontend juntos no terminal e mostrar as portas."
3. "Como faço para rodar o strace sem sudo e salvar um resumo das chamadas de sistema em arquivo?"
4. "Cria um script python para disparar perguntas concorrentes na API do RAG e medir o tempo de resposta."

**Sugestões aproveitadas:** O script start\_all.sh para subir os serviços; o comando strace -f -c para gerar o resumo estatístico; e a estrutura do script em Python para fazer as requisições concorrentes.

**Sugestões corrigidas ou rejeitadas:** Comandos com sudo sugeridos pela IA (optamos por rodar tudo em espaço de usuário); sugestão de testar concorrência muito alta que travava a máquina; e tabelas em markdown que desconfiguravam ao colar no documento.

**Erros encontrados nas respostas da IA:** Sugerir comandos que precisavam de root; achar que a GPU atendia as requisições ao mesmo tempo (quando na verdade ela atende em fila); e erros em dependências de pacotes.

**Como as respostas foram verificadas:** Tudo foi testado na prática no terminal do Ubuntu, conferindo as saídas dos comandos e os logs da aplicação.

**Distribuição das contribuições dos integrantes no uso de IA:** Apenas o integrante Bruno Saint Clair Silva Oliveira utilizou IA para auxiliar na execução e nos testes. Os outros integrantes não utilizaram IA nesta etapa.

## 15. Conclusão

A realização deste trabalho permitiu analisar o impacto da execução local de pipelines de IA generativa sobre o Sistema Operacional, resultando nos seguintes achados:

**Autonomia vs. Gargalo de Hardware:** A inferência local garante isolamento de dados e custo zero por token, porém é severamente limitada pela memória VRAM disponível e pelo processamento sequencial da GPU.

**Impacto da Concorrência:** O modelo de atendimento em fila do servidor de inferência degrada o tempo de resposta em  $3,73\times$  sob carga simultânea ( $N=4$ ), expondo o risco de timeouts na camada HTTP da aplicação.

**Gestão pelo Sistema Operacional:** O kernel do Linux lida eficientemente com a distribuição de centenas de threads ativas e primitivas de I/O, contudo, a saturação de VRAM força o transbordamento do contexto para a memória RAM física, aumentando consideravelmente o Resident Set Size (RSS) dos processos.

Em suma, a execução local de RAG é totalmente viável para cenários mono-usuário, mas exige GPUs com maior capacidade de VRAM ou arquiteturas de enfileiramento distribuído para suportar uso concorrente em ambientes de produção.

## 16. Referências